BNF and EBNF – Backus Naur Form

Computation and Discrete Structures III

Juan Marcos Caicedo Mejía — jmcaicedo@icesi.edu.co

Department of Computing and Intelligent Systems
Barberi Faculty of Engineering, Design and Applied Sciences
ICESI University

Material adapted from Professor Oscar Bedoya, Angela Villota and Andres Aristizabal

2026

Contents

1. BNF: Backus Naur Form

2. EBNF

2.1 Language and Syntax
2.2 EBNF Rules and Descriptions
2.3 Proving Symbols Match EBNF Rules
2.4 Syntax versus Semantics

Contents

1. BNF: Backus Naur Form

2. EBNF

Backus-Naur Form

Backus-Naur Form

The Backus-Naur Form is used to specify syntactic rules of many programming languages and natural languages: instead of using the symbol $\longrightarrow$ we use `::=` and enclose non-terminal symbols within <>.

BNF is frequently used to specify the syntax of programming languages such as Java and LISP; database languages such as SQL; and markup languages such as XML.

Backus-Naur Form (BNF)

It is used to specify syntactic rules of many programming languages. Instead of using ->, we use ::= and enclose non-terminal symbols within < >

$$
\begin{aligned}
S &\rightarrow aA \mid bB \\
A &\rightarrow aA \mid a \\
B &\rightarrow bB \mid b
\end{aligned}
\quad \Longrightarrow \quad
\begin{aligned}
\langle S \rangle &::= a\langle A \rangle \mid b\langle B \rangle \\
\langle A \rangle &::= a\langle A \rangle \mid a \\
\langle B \rangle &::= b\langle B \rangle \mid b
\end{aligned}
$$

Example 1

Consider the following grammar:

$$A \rightarrow Aa \mid a \mid AB$$

Its Backus-Naur Form is:

$$\langle A \rangle ::= \langle A \rangle a \mid a \mid \langle A \rangle \langle B \rangle$$

Example 2

Example 2. Consider the following CFG:

Grammar rules

$$
\begin{aligned}
S &\rightarrow NP\ VP \\
NP &\rightarrow \text{article noun} \\
VP &\rightarrow \text{verb noun} \\
\text{article} &\rightarrow \text{the} \\
\text{verb} &\rightarrow \text{eats} \\
\text{noun} &\rightarrow \text{dog} \mid \text{sausage}
\end{aligned}
$$

Identify the non-terminals and terminals in the grammar above.

Example 2

Example 2. Consider the following CFG:

Grammar rules

$$
\begin{aligned}
S &\rightarrow NP\ VP \\
NP &\rightarrow \text{article noun} \\
VP &\rightarrow \text{verb noun} \\
\text{article} &\rightarrow \text{the} \\
\text{verb} &\rightarrow \text{eats} \\
\text{noun} &\rightarrow \text{dog} \mid \text{sausage}
\end{aligned}
$$

Backus-Naur Form

$$
\begin{aligned}
\langle S \rangle &::= \langle NP \rangle \langle VP \rangle \\
\langle NP \rangle &::= \langle \text{article} \rangle \langle \text{noun} \rangle \\
\langle VP \rangle &::= \langle \text{verb} \rangle \langle \text{noun} \rangle \\
\langle \text{article} \rangle &::= \text{the} \\
\langle \text{verb} \rangle &::= \text{eats} \\
\langle \text{noun} \rangle &::= \text{dog} \mid \text{sausage}
\end{aligned}
$$

Each grammar rule maps directly to a BNF production. Non-terminals are enclosed in <>.

Example 3

The production of signed integers in decimal notation. (A **signed integer** is a natural number preceded by a plus or minus sign).

Backus-Naur Form

$$
\begin{aligned}
\langle \text{signed integer} \rangle &::= \langle \text{sign} \rangle \langle \text{integer} \rangle \\
\langle \text{sign} \rangle &::= + \mid - \\
\langle \text{integer} \rangle &::= \langle \text{digit} \rangle \mid \langle \text{digit} \rangle \langle \text{integer} \rangle \\
\langle \text{digit} \rangle &::= 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9
\end{aligned}
$$

Contents

1. BNF: Backus Naur Form

2. EBNF
   2.1 Language and Syntax
   2.2 EBNF Rules and Descriptions
   2.3 Proving Symbols Match EBNF Rules
   2.4 Syntax versus Semantics

Introduction

What is EBNF?

Introduction

What is EBNF?

Extended Backus-Naur Form (EBNF)

- **MetaLanguage**

Introduction

What is EBNF?

Extended Backus-Naur Form (EBNF)

- **MetaLanguage**
- Notation for formally describing syntax
  (how to write the linguistic features in a language).

Introduction

What is EBNF?

Extended Backus-Naur Form (EBNF)

- **MetaLanguage**
- Notation for formally describing syntax
  (how to write the linguistic features in a language).
- Also similar is the ability to name descriptions and reuse these names to build more complex structures.

Introduction

What is EBNF?

Extended Backus-Naur Form (EBNF)

- **MetaLanguage**
- Notation for formally describing syntax
  (how to write the linguistic features in a language).
- Also similar is the ability to name descriptions and reuse these names to build more complex structures.
- **Why EBNF?** EBNF stands for *Extended Backus-Naur Form*.

Contents

1. BNF: Backus Naur Form

2. EBNF
   2.1 Language and Syntax
   2.2 EBNF Rules and Descriptions
   2.3 Proving Symbols Match EBNF Rules
   2.4 Syntax versus Semantics

Language and Syntax

What did Backus do?

Language and Syntax

What did Backus do?

John Backus

- John Backus led the effort to develop **FORTRAN**. Later became a member of the **ALGOL** design committee.
- He invented a notation (based on the work of logician Emil Post) that was simple, precise, and powerful enough to describe the syntax of any programming language.
- Using this notation, a programmer or compiler can determine whether a program is syntactically correct: whether it adheres to the grammar and punctuation rules of the programming language.

Language and Syntax

What did Backus do?

John Backus

- John Backus led the effort to develop **FORTRAN**. Later became a member of the **ALGOL** design committee.
- He invented a notation (based on the work of logician Emil Post) that was simple, precise, and powerful enough to describe the syntax of any programming language.
- Using this notation, a programmer or compiler can determine whether a program is syntactically correct: whether it adheres to the grammar and punctuation rules of the programming language.

And who popularized this notation?

Language and Syntax

What did Backus do?

John Backus

- John Backus led the effort to develop **FORTRAN**. Later became a member of the **ALGOL** design committee.
- He invented a notation (based on the work of logician Emil Post) that was simple, precise, and powerful enough to describe the syntax of any programming language.
- Using this notation, a programmer or compiler can determine whether a program is syntactically correct: whether it adheres to the grammar and punctuation rules of the programming language.

And who popularized this notation?

Peter Naur

- Peter Naur.
- As an editor of the ALGOL report, he popularized this notation by using it to describe the complete syntax of ALGOL.
- In Backus and Naur's honor this notation is called Backus-Naur Form (BNF).
- And the **E??** comes from the word **extended**.

Contents

1. BNF: Backus Naur Form

2. EBNF
2.1 Language and Syntax
2.2 EBNF Rules and Descriptions
2.3 Proving Symbols Match EBNF Rules
2.4 Syntax versus Semantics

EBNF Rules and Descriptions

What is an EBNF description?

EBNF Rules and Descriptions

What is an EBNF description?

EBNF Description

- Is an unordered list of EBNF rules.
- Each EBNF rule has three parts: *left* separator *right* ;
  - The left-hand side (LHS), or *left* is a non-terminal, similar to the *head* in a production rule.
  - The separator is the designed character separating these two sides. Usually, it may be the symbol ::=, =, or ⇐.
  - The right-hand side (RHS), or *right* is a combination of non-terminals, terminals, and symbols. Similar to the body in a production rule.
  - The semicolon is the termination symbol.
- Non-terminals are enclosed in angular brackets as in ⟨*option*⟩.
- Terminal symbols may be enclosed in quotes.
- The RHS may contain terminals, non-terminals and combinations of the four control forms (sequence, choice, option repetition).

EBNF Rules and Descriptions

Which are the four control forms of RHS?

EBNF Rules and Descriptions

Which are the four control forms of RHS?

Control Forms

- **Sequence:** Items appear left-to-right; their order is important.
- **Choice:** Alternatives separated by | (pipe). One item is chosen; their order is unimportant.
- **Option:** The optional item is marked by ?; it can be included or discarded.
- **Repetition:** Represented by $*$ (zero or more times). S^+ means S appears at least once.

Languages described in BNF offer significant advantages for the design of syntactic analyzers in compilers.

EBNF Rules and Descriptions

Example

EBNF Description: *integer*

$$
\begin{array}{lcl}
\langle \textit{digit} \rangle & ::= & \texttt{'0'|'1'|'2'|'3'|'4'|'5'|'6'|'7'|'8'|'9'}\ ; \\
\langle \textit{symbol} \rangle & ::= & \texttt{'+'|'-'} \\
\langle \textit{integer} \rangle & ::= & \langle \textit{symbol} \rangle ?\ \langle \textit{digit} \rangle\ \langle \textit{digit} \rangle *\ ;
\end{array}
$$

Contents

1. BNF: Backus Naur Form

2. EBNF
2.1 Language and Syntax
2.2 EBNF Rules and Descriptions
2.3 Proving Symbols Match EBNF Rules
2.4 Syntax versus Semantics

Proving Symbols Match EBNF Rules

How to prove that a symbol is legal according to some EBNF rule?

Proving Symbols Match EBNF Rules

How to prove that a symbol is legal according to some EBNF rule?

Proof Criteria

- We must match all its characters with all the items in the EBNF rule, according to that rule's description.
- If there is an exact match we classify the symbol as **legal** according to that EBNF description and say it **matches**.
- Otherwise we classify the symbol as **illegal** and say it **doesn't match**.

Proving Symbols Match EBNF Rules

Example

EBNF Description: *integer*

$$
\begin{array}{lcl}
\langle \textit{digit} \rangle & ::= & \texttt{'0'|'1'|'2'|'3'|'4'|'5'|'6'|'7'|'8'|'9'}\ ; \\
\langle \textit{symbol} \rangle & ::= & \texttt{'+'|'-'} \\
\langle \textit{integer} \rangle & ::= & \langle \textit{symbol} \rangle ?\ \langle \textit{digit} \rangle\ \langle \textit{digit} \rangle *\ ;
\end{array}
$$

- We can prove that the symbol `+142` matches the integer EBNF rule.
- We can easily prove that `1,024` is an illegal integer.

Proving Symbols Match EBNF Rules

What is a tabular proof?

Proving Symbols Match EBNF Rules

What is a tabular proof?

Tabular Proof

- A tabular proof is a more formal demonstration that a symbol matches an EBNF description.
- The first line in a tabular proof is always the name of the EBNF rule that specifies the syntactic structure we are trying to match the symbol against.
- The last line is the symbol we are matching.
- Each line is derived from the previous according to one of the following rules:
  1. Replace a name (LHS) by its definition (RHS)
  2. Choose an alternative
  3. Determine whether to include or discard an option
  4. Determine the number of times to repeat
- Combining rules 1 and 2 (1&2) simplifies our proofs by allowing us, in a single step, to replace a left-hand side by one of the alternatives in its right-hand side.

Proving Symbols Match EBNF Rules

Example

A tabular proof and its derivation tree that shows the symbol +142 matches the integer EBNF rule.

Step	Rule / Action	Sentential form
1	*Start*	⟨*integer*⟩
2	*Expand* ⟨*integer*⟩	⟨*symbol*⟩?⟨*digit*⟩ ⟨*digit*⟩∗
3	*Include optional symbol*	⟨*symbol*⟩ ⟨*digit*⟩ ⟨*digit*⟩∗
4	*Expand* ⟨*symbol*⟩	`'+'` ⟨*digit*⟩ ⟨*digit*⟩∗
5	*Expand first* ⟨*digit*⟩	`'+' '1'` ⟨*digit*⟩∗
6	*Realize repetition as two digits (choose two)*	`'+' '1'` ⟨*digit*⟩ ⟨*digit*⟩
7	*Replace first repeated* ⟨*digit*⟩	`'+' '1' '4'` ⟨*digit*⟩
8	*Replace last* ⟨*digit*⟩	`'+' '1' '4' '2'`
9	*End repetition / yield terminal string*	$+142$

Contents

1. BNF: Backus Naur Form

2. EBNF
2.1 Language and Syntax
2.2 EBNF Rules and Descriptions
2.3 Proving Symbols Match EBNF Rules
2.4 Syntax versus Semantics

Syntax versus Semantics

What to take into account?

Syntax versus Semantics

What to take into account?

Syntax vs. Semantics

- EBNF descriptions specify only **syntax**: the form in which something is written.
- They do not specify **semantics**: the meaning of what is written.
- Syntax = Form
- Semantics = Meaning
- Two semantic issues are important in programming languages:
  - Can different symbols have the same meaning?
  - Can one symbol have different meanings?
- Different symbols can have the same meaning and one symbol can have different meanings depending on its context.

Backus-Naur Form (BNF)

Definition

BNF is a **meta-language**: a formal notation used to specify the syntactic rules of programming and natural languages. Instead of $\rightarrow$, we use `::=` and enclose **non-terminals** within `<` `>`.

Backus–Naur Form (BNF)

Definition

BNF is a **meta-language**: a formal notation used to specify the syntactic rules of programming and natural languages. Instead of $\rightarrow$, we use `::=` and enclose **non-terminals** within `<` `>`.

Structure of a BNF rule:

$$\underbrace{\langle \text{name} \rangle}_{\text{LHS (non-terminal)}} \quad ::= \quad \underbrace{\langle A \rangle \ \langle B \rangle \mid \langle C \rangle}_{\text{RHS (terminals, non-terminals, choices)}}$$

Backus–Naur Form (BNF)

Definition

BNF is a **meta-language**: a formal notation used to specify the syntactic rules of programming and natural languages. Instead of $\rightarrow$, we use `::=` and enclose **non-terminals** within `<` `>`.

Structure of a BNF rule:

$$\underbrace{\langle \text{name} \rangle}_{\text{LHS (non-terminal)}} \quad ::= \quad \underbrace{\langle A \rangle \ \langle B \rangle \mid \langle C \rangle}_{\text{RHS (terminals, non-terminals, choices)}}$$

BNF only supports two operations on the RHS:

- **Sequence** — items appear left-to-right in order

Backus–Naur Form (BNF)

Definition

BNF is a **meta-language**: a formal notation used to specify the syntactic rules of programming and natural languages. Instead of $\rightarrow$, we use `::=` and enclose **non-terminals** within `<` `>`.

Structure of a BNF rule:

$$\underbrace{\langle \text{name} \rangle}_{\text{LHS (non-terminal)}} \quad ::= \quad \underbrace{\langle A \rangle \ \langle B \rangle \mid \langle C \rangle}_{\text{RHS (terminals, non-terminals, choices)}}$$

BNF only supports two operations on the RHS:

- **Sequence** — items appear left-to-right in order
- **Choice** (|) — pick one alternative

Backus–Naur Form (BNF)

Definition

BNF is a **meta-language**: a formal notation used to specify the syntactic rules of programming and natural languages. Instead of $\rightarrow$, we use `::=` and enclose **non-terminals** within `<` `>`.

Structure of a BNF rule:

$$\underbrace{\langle \text{name} \rangle}_{\text{LHS (non-terminal)}} \quad ::= \quad \underbrace{\langle A \rangle \ \langle B \rangle \mid \langle C \rangle}_{\text{RHS (terminals, non-terminals, choices)}}$$

BNF only supports two operations on the RHS:

- **Sequence** — items appear left-to-right in order
- **Choice** (|) — pick one alternative

Limitation

Repetition and optional elements **cannot** be expressed directly — they require auxiliary recursive rules.

Extended BNF (EBNF) — The Four Control For

EBNF adds **two new operators** to BNF's sequence and choice:

1. Sequence

Items appear **left-to-right**; order matters.

```
A B C
```

Extended BNF (EBNF) — The Four Control For

EBNF adds **two new operators** to BNF's sequence and choice:

1. Sequence

Items appear **left-to-right**; order matters.

```
A B C
```

2. Choice

Alternatives separated by | ; order does *not* matter.

```
A | B | C
```

Extended BNF (EBNF) — The Four Control For

EBNF adds **two new operators** to BNF's sequence and choice:

1. Sequence

Items appear **left-to-right**; order matters.

```
A B C
```

2. Choice

Alternatives separated by | ; order does *not* matter.

```
A | B | C
```

3. Option (new in EBNF)

The item is **optional**: zero or one occurrence.

```
A?
```

Extended BNF (EBNF) — The Four Control For

EBNF adds **two new operators** to BNF's sequence and choice:

1. Sequence

Items appear **left-to-right**; order matters.

```
A B C
```

2. Choice

Alternatives separated by | ; order does *not* matter.

```
A | B | C
```

3. Option (new in EBNF)

The item is **optional**: zero or one occurrence.

```
A?
```

4. Repetition (new in EBNF)

Zero or more occurrences: `{A}`
One or more occurrences: $\{A\}^+$

Extended BNF (EBNF) — The Four Control For

EBNF adds **two new operators** to BNF's sequence and choice:

1. Sequence

Items appear **left-to-right**; order matters.

`A B C`

2. Choice

Alternatives separated by `|`; order does *not* matter.

`A | B | C`

3. Option (new in EBNF)

The item is **optional**: zero or one occurrence.

`A?`

4. Repetition (new in EBNF)

Zero or more occurrences: `{A}`
One or more occurrences: `{A}`$^{+}$

Also in EBNF: rules end with `;` · separator can be `=`, `::=`, or `:` · terminals may be enclosed in quotes.

BNF vs. EBNF — Signed Integers

Goal: describe signed decimal integers, e.g. −142, +7, 0.

In BNF

$$
\begin{aligned}
\langle \text{sint} \rangle &::= \langle \text{sign} \rangle \langle \text{int} \rangle \\
\langle \text{sign} \rangle &::= + \mid - \\
\langle \text{int} \rangle &::= \langle \text{digit} \rangle \\
&\quad \mid \langle \text{digit} \rangle \langle \text{int} \rangle \\
\langle \text{digit} \rangle &::= 0 \mid 1 \mid \cdots \mid 9
\end{aligned}
$$

Repetition ⇒ **recursive rule** needed.
Sign ⇒ **always required** (no option).

BNF vs. EBNF — Signed Integers

Goal: describe signed decimal integers, e.g. −142, +7, 0.

In BNF

$$
\begin{aligned}
\langle \text{sint} \rangle &::= \langle \text{sign} \rangle \langle \text{int} \rangle \\
\langle \text{sign} \rangle &::= + \mid - \\
\langle \text{int} \rangle &::= \langle \text{digit} \rangle \\
&\quad \mid \langle \text{digit} \rangle \langle \text{int} \rangle \\
\langle \text{digit} \rangle &::= 0 \mid 1 \mid \cdots \mid 9
\end{aligned}
$$

Repetition ⇒ **recursive rule** needed.
Sign ⇒ **always required** (no option).

In EBNF

$$
\begin{aligned}
\langle \text{integer} \rangle &::= \langle \text{symbol} \rangle^{?} \langle \text{digit} \rangle \{ \langle \text{digit} \rangle \} \\
\langle \text{symbol} \rangle &::= - \\
\langle \text{digit} \rangle &::= 0 \mid 1 \mid \cdots \mid 9
\end{aligned}
$$

? ⇒ sign is **optional**.
`{digit}` ⇒ **repetition**, no recursion.

BNF vs. EBNF — Signed Integers

Goal: describe signed decimal integers, e.g. −142, +7, 0.

In BNF

$$
\begin{aligned}
\langle \text{sint} \rangle &::= \langle \text{sign} \rangle \langle \text{int} \rangle \\
\langle \text{sign} \rangle &::= + \mid - \\
\langle \text{int} \rangle &::= \langle \text{digit} \rangle \\
&\quad \mid \langle \text{digit} \rangle \langle \text{int} \rangle \\
\langle \text{digit} \rangle &::= 0 \mid 1 \mid \cdots \mid 9
\end{aligned}
$$

Repetition ⇒ **recursive rule** needed.
Sign ⇒ **always required** (no option).

In EBNF

$$
\begin{aligned}
\langle \text{integer} \rangle &::= \langle \text{symbol} \rangle^{?} \langle \text{digit} \rangle \{ \langle \text{digit} \rangle \} \\
\langle \text{symbol} \rangle &::= - \\
\langle \text{digit} \rangle &::= 0 \mid 1 \mid \cdots \mid 9
\end{aligned}
$$

? ⇒ sign is **optional**.
`{digit}` ⇒ **repetition**, no recursion.

Key insight

EBNF eliminates the recursive `<int>` rule entirely. **Fewer rules, same expressive power.**

BNF vs. EBNF — Summary

Feature	BNF	EBNF
Separator	`::=`	`::=`
Termination symbol	none	`;`
Non-terminal notation	`<name>`	`<name>`
Terminal notation	plain text	plain text or quoted

BNF vs. EBNF — Summary

Feature	BNF	EBNF
Separator	`::=`	`::=`
Termination symbol	none	`;`
Non-terminal notation	`<name>`	`<name>`
Terminal notation	plain text	plain text or quoted
Sequence	✓	✓
Choice (|)	✓	✓

BNF vs. EBNF — Summary

Feature	BNF	EBNF
Separator	`::=`	`::=`
Termination symbol	none	`;`
Non-terminal notation	`<name>`	`<name>`
Terminal notation	plain text	plain text or quoted
Sequence	✓	✓
Choice (`|`)	✓	✓
Option (?)	× (needs rule)	✓
Repetition (`{}`)	× (needs recursion)	✓

BNF vs. EBNF — Summary

Feature	BNF	EBNF
Separator	`::=`	`::=`
Termination symbol	none	`;`
Non-terminal notation	`<name>`	`<name>`
Terminal notation	plain text	plain text or quoted
Sequence	✓	✓
Choice (`|`)	✓	✓
Option (?)	× (needs rule)	✓
Repetition (`{}`)	× (needs recursion)	✓

Real-world use

EBNF is used to specify the full syntax of languages like **Java**, **SQL**, **XML**, and **Python** (`docs.python.org/3/reference/grammar.html`).